UG PROJECT WORK - PHASE I - REVIEW I

Bayesian Belief Network-Driven Proactive Fault Tolerance for Microservice Placement in Fog Computing

PRESENTED BY

Mohit R — 3122235002072 Rahul Akshath Rajendran — 3122235002095

PROJECT GUIDE

Dr. Sabireen H, Assistant Professor

Department of Information Technology
SSN College of Engineering, Kalavakkam

20 August 2026

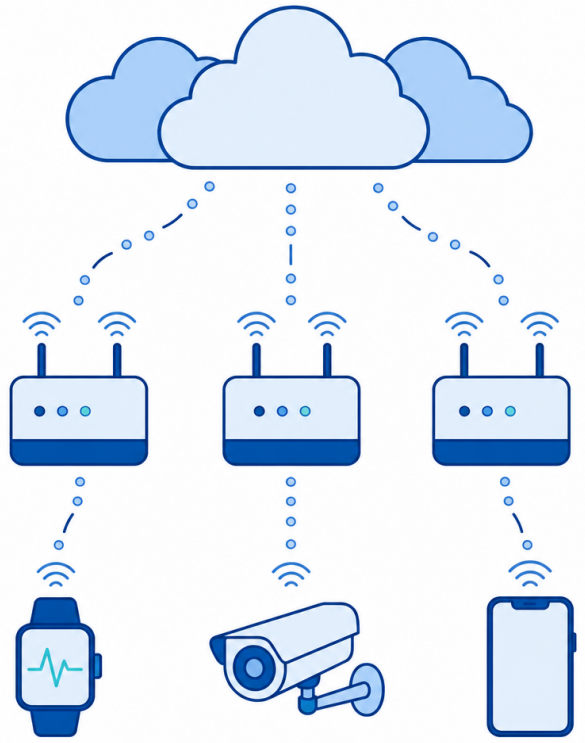
Problem Statement

Existing fog fault-tolerance mechanisms are mainly reactive and recover only after node failures occur. Proactive methods based on fixed thresholds are unreliable because fog-node health is noisy and partially observable. There is no mechanism that estimates the probability of a fog node failing within a given time. Therefore, a Bayesian Belief Network is needed to predict failure risk and guide placement, migration, and redundancy decisions.

Introduction

Why fog computing is used

- IoT applications such as cardiac monitoring and video surveillance must react within tens of milliseconds.
- Sending every reading to a distant cloud data centre cannot meet that deadline.
- Fog computing runs the application modules on gateways, routers and base stations, close to the sensor.



Why fog nodes are unreliable

- A fog node is ordinary hardware in an uncontrolled location, sharing a limited uplink and often on battery.
- Mobile users move from one node to another, so the load each node carries keeps changing.
- Such nodes fail far more often than data-centre servers, and one failure can break a critical deadline.

Why the failure can be anticipated

- Nodes seldom fail without warning. Utilisation rises, queues grow, links deteriorate and energy drains first.
- This leaves a short window in which a microservice can be moved before the node stops responding.
- This project predicts failure inside that window and uses it to place, migrate and replicate modules.

Literature Survey

Paper	Approach	Main finding
[1] "Dynamic fault tolerance aware scheduling for healthcare system on fog computing" 2021	Weighted round-robin scheduling, checkpointing when CPU use passes 0.2, primary-backup replication	Load-aware scheduling raises throughput; adaptive checkpointing costs less than fixed intervals
[2] "Fault tolerance mechanism for software application through fog computing as middleware" 2016	Fog middleware that polls the server with heartbeats and switches to a replicated backup	Failover can be made invisible to the user; recovered three of four test scenarios
[3] "Optimized fault-tolerant placement of microservices in distributed fog networks" 2025	FTAPA: load-balanced primary node, a pre-selected backup, migration on a health threshold	Moving a microservice before its host fails measurably reduces service downtime
[4] "Using Bayesian networks to predict software defects and reliability" 2007	Bayesian networks predicting software defects from causal factors and measurement data	Expert judgement and measured data can be combined; used at Motorola, Siemens and Philips
[5] "Overview on Bayesian networks applications for dependability, risk analysis and maintenance areas" 2012	Survey of ten years of Bayesian network use in dependability, risk and maintenance	Bayesian networks suit heterogeneous variables better than fault trees and Markov chains
[6] "Bayesian networks in environmental modelling" 2011	Review of Bayesian networks applied to environmental modelling under sparse observation	Inference still produces a usable result when part of the evidence is missing

Research Gap

- **[1] “Dynamic fault tolerance aware scheduling for healthcare system on fog computing”.** The CPU threshold is fixed and ignores the workload, and a backup is stored for every task.
- **[2] “Fault tolerance mechanism for software application through fog computing as middleware”.** Recovery begins only after the server stops answering, and a single backup is kept.
- **[3] “Optimized fault-tolerant placement of microservices in distributed fog networks”.** The fault threshold is hard to set, and a backup node is reserved for every microservice.
- **[4] “Using Bayesian networks to predict software defects and reliability”.** Built for software processes, not running infrastructure, and the tables are never updated online.
- **[5] “Overview on Bayesian networks applications for dependability, risk analysis and maintenance areas”.** Covers industrial plants, not fog computing, where telemetry is often incomplete.
- **[6] “Bayesian networks in environmental modelling”.** No decision has to be taken within a deadline, so nothing is placed or migrated in real time.
- **Common gap:** none of them estimates the probability that a fog node will fail, or uses such an estimate to decide placement, migration and redundancy.

Objectives

Objective 1: Estimate the failure of a fog node as a probability

- Infer, for every node, the probability of failure within a look-ahead horizon Δt.
- Use whatever telemetry is available at the time, and marginalise over the rest.
- Assessed by precision, recall, calibration and the lead time before the failure.

Objective 2: Use that probability to place and migrate microservices

- Score candidate nodes on latency, energy and failure risk before a module is placed.
- Move a running module only when the expected loss of staying exceeds the cost of moving.
- Assessed by loop delay, deadline violations and downtime against the baselines.

Objective 3: Allocate redundancy by risk and application criticality

- Set each module's redundancy from its predicted risk and its application's criticality.
- Assessed by the downtime obtained for a given amount of reserved standby capacity.

Proposed Novelties

Novelty 1 (Objective 1): a causal Bayesian Belief Network over node health

- Failure is expressed as a probability, not a threshold crossing or a binary alarm.
- Evidence missing at a decision epoch is marginalised out, so a prediction is still produced.
- The network is causal, so it also reports which condition is behind the estimate.

Novelty 2 (Objective 2): expected-loss migration in place of a fixed threshold

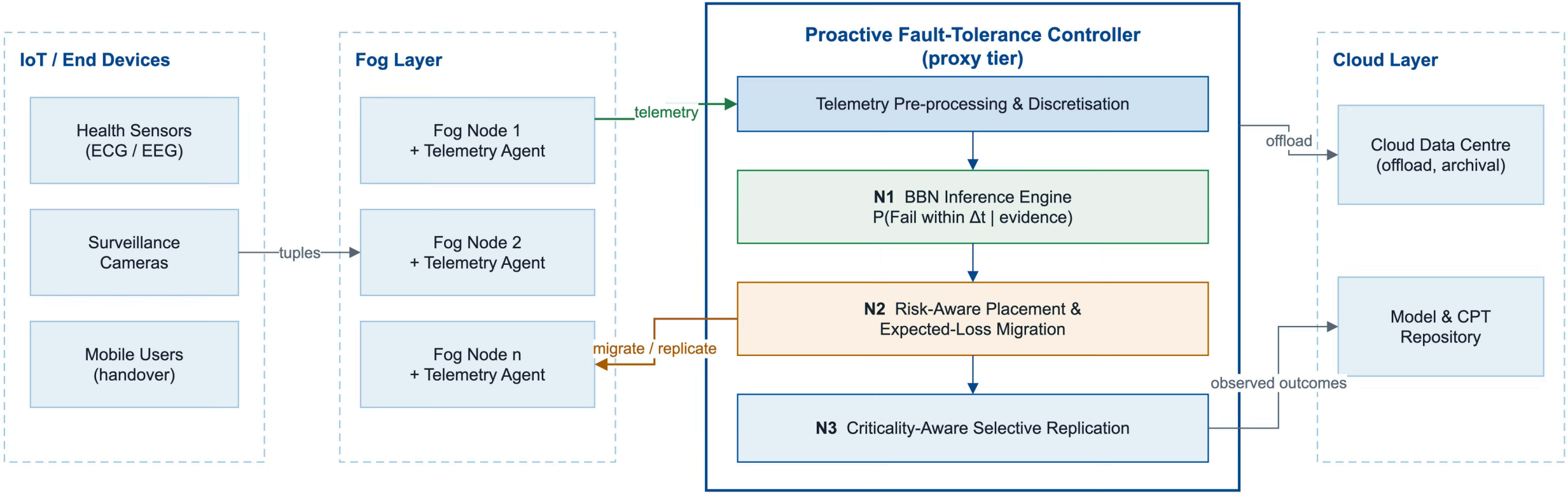
- The probability enters a utility over latency, energy and risk that ranks candidate nodes.
- A module is moved only when the expected loss from staying exceeds the cost of moving it.
- The effective threshold follows from the cost ratio, so it differs from module to module.

Novelty 3 (Objective 3): redundancy that follows risk and criticality

- Redundancy is a resource to be spent, not a backup reserved for every module.
- Modules go into warm-standby, checkpoint-only or no-redundancy tiers by risk × criticality.
- Tiers are recomputed as the probabilities change, so capacity follows the risk over time.

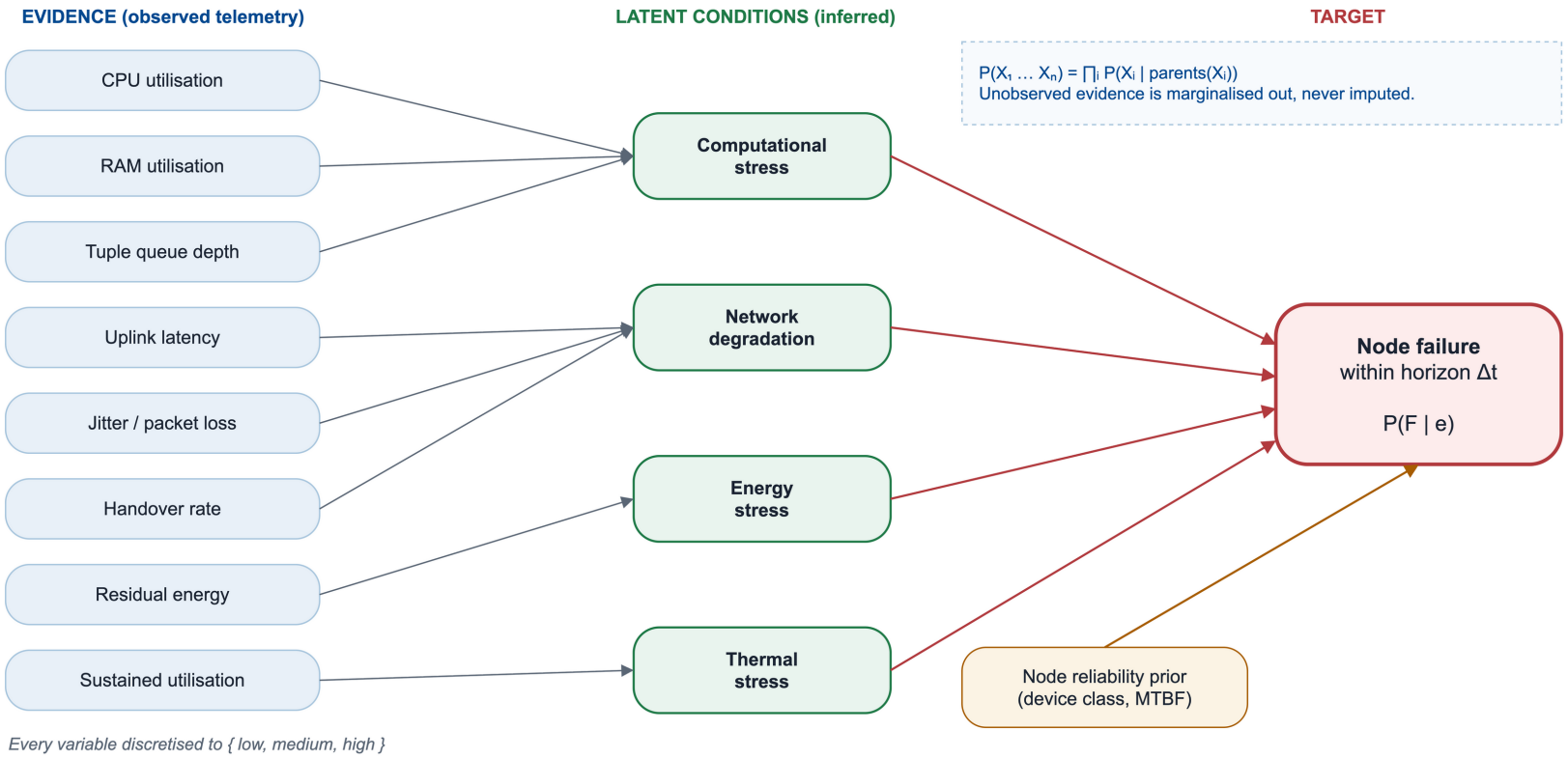
Methodology — System Architecture

The Proactive Fault-Tolerance Controller sits at the proxy tier, above a cluster of fog nodes



Telemetry flows up from every fog node; placement, migration and replication decisions flow back down.

Methodology — The Bayesian Belief Network



Every variable discretised to { low, medium, high }

Why a Bayesian Belief Network

- ~1.6 M joint entries collapse to a few hundred, factorised
- Exact inference; missing evidence is marginalised out
- Causal — it names the condition driving the risk

Where the numbers come from

- Structure from domain knowledge, not learned from data
- Initial CPTs seeded from the reliability literature
- Refined online by Dirichlet parameter updating

Methodology — Decision Rules



1. PLACEMENT — SCORE EVERY CANDIDATE NODE

$$U(m,n) = \alpha\,(1 - \text{latency}) + \beta\,(1 - \text{energy}) + \gamma\,(1 - P(F \mid e))$$

- Risk enters the score directly; weights are set per application class



2. MIGRATION — MOVE ONLY WHEN IT PAYS

$$E[\text{loss} \mid \text{stay}] = P(F \mid e) \cdot C_{\text{fail}}$$

$$E[\text{loss} \mid \text{migrate}] = C_{\text{migrate}}$$

- Migrate when $P(F \mid e) \cdot C_{\text{fail}} > C_{\text{migrate}}$ — no threshold is chosen, it falls out of the cost ratio

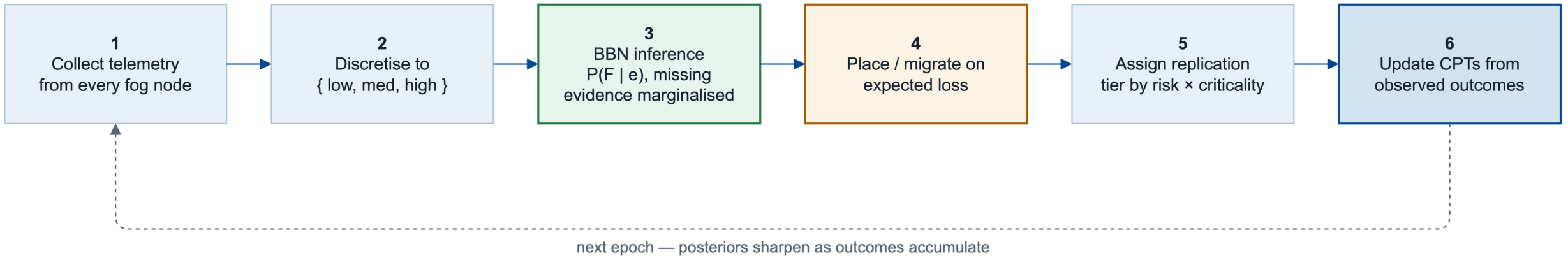


3. REPLICATION — SPEND REDUNDANCY WHERE RISK MEETS CRITICALITY

$$R(m) = P(F \mid e) \times \kappa(m) \longrightarrow \text{warm-standby} \mid \text{checkpoint-only} \mid \text{no redundancy}$$

- Tiers are recomputed each epoch, so reserved capacity follows the risk over time

Methodology — Execution Flow per Epoch



What runs at each decision epoch

- Steps 1–3 are inference: telemetry in, a posterior P(F | e) per node out
- Steps 4–5 are control: placement, migration and replication act on that posterior
- Step 6 closes the loop — outcomes update the CPTs, so the model sharpens as the run proceeds

Dataset Description

What the dataset contains

- edgeResources-melbCBD.csv gives 131 real edge sites, with latitude, longitude, block, level and parent.
- usersLocation-melbCBD_1.csv gives 87 recorded user positions; five synthetic traces add 100 each.
- ApplicationConfig.json gives ten microservice profiles with MIPS, network load and deadline.

How iFogSim2 builds the devices

- Each row becomes a FogDevice: createFogDevice() builds a processing element, power host and tier.
- The four levels give 1 cloud data centre, 12 block proxy servers, 118 gateways and mobile devices.
- Sensors (ECG, EEG, camera) and actuators (display, PTZ) attach at level 3 and emit the tuples.

Tier (level)	What it represents	MIPS	RAM (MB)	Uplink BW	Busy / Idle W
Cloud (0)	Cloud data centre, 1 node	44 800	40 000	100	1 648 / 1 332
Proxy (1)	Block proxy servers, 12 nodes	2 800	4 000	10 000	107.34 / 83.43
Gateway (2)	Cell-site gateways, 118 nodes	2 800	4 000	10 000	107.34 / 83.43
Mobile (3)	Phones, cameras, ECG and EEG nodes	200 - 1 000	1 024 - 2 048	10 000	87.53 / 82.44

Simulating Conditions and Failures in iFogSim2

How different conditions are produced

- Load is varied by changing how many modules sit on a node and the MIPS each of them needs.
- Network conditions come from the configured link latency and the queueing delay on the uplink.
- Mobility traces move users between gateways, so the load each node carries changes during a run.

How failures are introduced

- No public failure trace exists for such a topology, so a harness injects them into iFogSim2.
- A hazard function over the simulator's true load, energy and link state decides when a node dies.
- The network never sees that state. It sees only sampled, discretised, sometimes missing telemetry.

Cases simulated in the evaluation

- A baseline run with no failures, to measure the normal loop delay and energy use.
- Single-node failures at the proxy and gateway tiers, under light load and under heavy load.
- Runs with telemetry withheld, and runs with more users, so handovers change the load.

From Simulator State to BBN Evidence

- Every evidence variable is computed from a quantity that iFogSim2 already tracks during a run.

Evidence variable	Quantity in iFogSim2 it is derived from	Latent condition it feeds
CPU utilisation	Allocated MIPS against host PE capacity, per scheduling interval	Computational stress
RAM utilisation	RamProvisionerSimple allocation against available RAM	Computational stress
Tuple queue depth	Length of the pending tuple queue on the device	Computational stress
Uplink latency and jitter	Configured link latency plus queueing delay on the uplink	Network degradation
Residual energy	FogDevice energy consumption against a configured budget	Energy stress
Handover rate	Parent reassignments per window, from LocationHandler	Network degradation
Node reliability prior	Device class (cloud, proxy, gateway, mobile) and its MTBF	Parent of the target

- All variables are binned into low, medium and high, using quantiles of the baseline runs.
- Thermal stress is inferred from sustained utilisation, as iFogSim2 has no temperature sensor.

Evaluation Plan, Baselines and Work Plan

Evaluation metrics

Prediction quality

- Precision, recall and F1 within Δt
- Calibration against observed frequency
- Lead time before the actual failure
- Robustness when telemetry is withheld

System quality

- Loop delay and deadline-violation ratio
- Energy across cloud, fog and device tiers
- Service downtime per injected failure
- Reserved standby capacity as overhead

Baselines for comparison

1. Cloud-only placement
2. iFogSim2 edgeward placement, no fault tolerance
3. Reactive checkpointing and replication [1]
4. Threshold-based proactive migration, FTAPA [3]

- Baseline 4 separates probabilistic inference from simply acting early.

Work plan

- Done: survey, iFogSim2 set up, baseline runs, design
- Next: build and validate the BBN and its CPTs
- Next: inference engine and fault-injection harness
- Next: placement, migration and replication policies
- Next: comparative evaluation against the baselines

References

[1] H. T. Rajab and M. F. Younis, “Dynamic fault tolerance aware scheduling for healthcare system on fog computing,” Iraqi Journal of Science, vol. 62, no. 1, pp. 308–318, 2021.

[2] M. H. Naim, J. M. Zain and K. A. Jalil, “Fault tolerance mechanism for software application through fog computing as middleware,” Int. J. Computing and Digital Systems, vol. 5, no. 3, 2016.

[3] S. Gupta and N. Singh, “Optimized fault-tolerant placement of microservices in distributed fog networks,” IJDIIC, vol. 4, no. 4, pp. 52–69, 2025.

[4] N. Fenton, M. Neil and D. Marquez, “Using Bayesian networks to predict software defects and reliability,” Proc. MMR07, Glasgow, 2007.

[5] P. Weber, G. Medina-Oliva, C. Simon and B. Iung, “Overview on Bayesian networks applications for dependability, risk analysis and maintenance areas,” Eng. Appl. Artif. Intell., vol. 25, no. 4, pp. 671–682, 2012.

[6] P. A. Aguilera et al., “Bayesian networks in environmental modelling,” Environmental Modelling & Software, vol. 26, no. 12, pp. 1376–1388, 2011.

[7] R. Mahmud, S. Pallewatta, M. Goudarzi and R. Buyya, “iFogSim2: An extended iFogSim simulator,” J. Systems and Software, vol. 190, art. 111351, 2022.

[8] H. Gupta, A. V. Dastjerdi, S. K. Ghosh and R. Buyya, “iFogSim: A toolkit for modeling and simulation of resource management techniques,” Software: Practice and Experience, vol. 47, no. 9, pp. 1275–1296, 2017.

Thank You

Mohit R and Rahul Akshath Rajendran

Guided by Dr. Sabireen H, Department of Information Technology